

Nghiên Cứu Lý Thuyết API Testing

Seminar W4 — Group 3

Môn học: Kiểm thử phần mềm

Trường: HCMUS

Ngày tạo: 2026-07-04

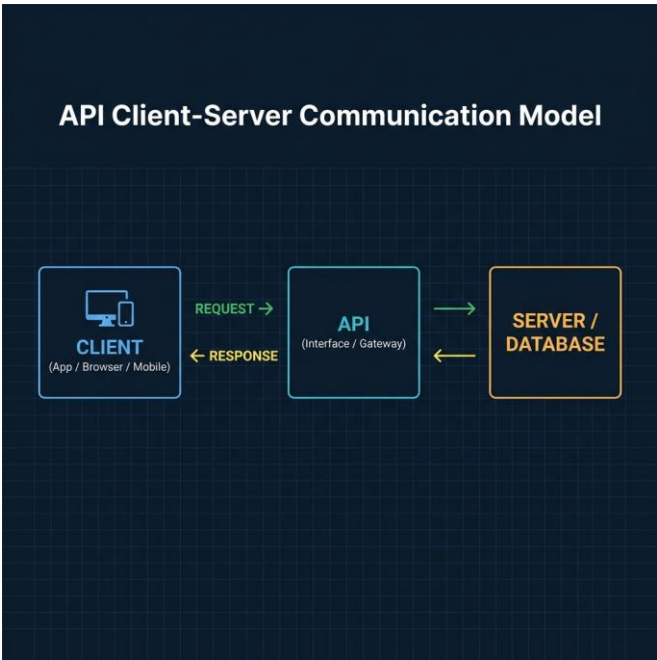
Mục lục

1. [Khái niệm API](#)
 2. [Kiến trúc & Giao tiếp API](#)
 3. [Các loại API](#)
 4. [Cấu trúc HTTP Request](#)
 5. [Cấu trúc HTTP Response](#)
 6. [Các loại HTTP Methods](#)
 7. [HTTP Status Codes](#)
 8. [Phân biệt API có Authentication và không có Authentication](#)
 9. [Các loại Authentication trong API](#)
 10. [Phân loại API Testing](#)
 11. [Checklist kiểm thử API](#)
 12. [Công cụ kiểm thử API phổ biến](#)
-

1. Khái niệm API

1.1 Định nghĩa

API (Application Programming Interface) là tập hợp các quy tắc, giao thức và định nghĩa cho phép các ứng dụng phần mềm khác nhau giao tiếp và trao đổi dữ liệu với nhau. API đóng vai trò như một "trung gian" (intermediary) — nhận yêu cầu từ client, xử lý tại server và trả về kết quả.



1.2 Vai trò của API

Vai trò	Mô tả
Tích hợp hệ thống	Cho phép các hệ thống khác nhau kết nối và chia sẻ dữ liệu
Trừu tượng hóa	Ẩn đi sự phức tạp của back-end, chỉ expose những gì cần thiết
Tái sử dụng	Một API có thể phục vụ nhiều loại client (web, mobile, desktop)
Bảo mật	Kiểm soát quyền truy cập vào dữ liệu và chức năng hệ thống
Mở rộng	Cho phép bên thứ ba xây dựng sản phẩm trên nền tảng hiện có

1.3 Các khái niệm liên quan

- **Endpoint:** URL cụ thể mà API lắng nghe yêu cầu, ví dụ:
`https://api.example.com/users`
 - **Resource:** Đối tượng hoặc dữ liệu mà API quản lý (ví dụ: user, product, order)
 - **Payload / Body:** Dữ liệu được gửi hoặc nhận trong một request/response
 - **Statelessness:** Nguyên tắc mỗi request phải tự chứa đủ thông tin cần thiết; server không lưu trạng thái phiên làm việc
 - **Base URL:** Phần URL gốc dùng chung cho tất cả endpoint, ví dụ:
`https://api.example.com/v1`
 - **API Contract:** Thỏa thuận về schema, format, và hành vi giữa client và server (thường được mô tả bằng OpenAPI/Swagger)
-

2. Kiến trúc & Giao tiếp API

2.1 Mô hình Client-Server

API hoạt động theo mô hình **Request-Response**:

1. **Client** gửi HTTP Request đến một Endpoint
2. **API Server** xử lý yêu cầu, truy vấn database/business logic
3. **Server** trả về HTTP Response với dữ liệu hoặc thông báo lỗi
4. **Client** đọc response và hiển thị / xử lý tiếp

2.2 Nguyên tắc Statelessness (REST)

- Mỗi request hoàn toàn **độc lập**
 - Server **không lưu** trạng thái (session) giữa các request
 - Mọi thông tin cần thiết (token, dữ liệu) phải được gửi **trong mỗi request**
 - Ưu điểm: dễ scale (horizontal scaling), dễ debug
-

3. Các loại API

3.1 Phân loại theo kiến trúc

Loại API	Giao thức	Định dạng dữ liệu	Đặc điểm nổi bật
REST	HTTP/HTTPS	JSON, XML	Đơn giản, phổ biến nhất, resource-based
SOAP	HTTP, SMTP, TCP	XML only	Nghiêm ngặt, có chuẩn WS-Security
GraphQL	HTTP/HTTPS	JSON	Client kiểm soát dữ liệu trả về
gRPC	HTTP/2	Protocol Buffers	Hiệu năng cao, phù hợp microservices
WebSocket	WebSocket	JSON, binary	Kết nối hai chiều real-time

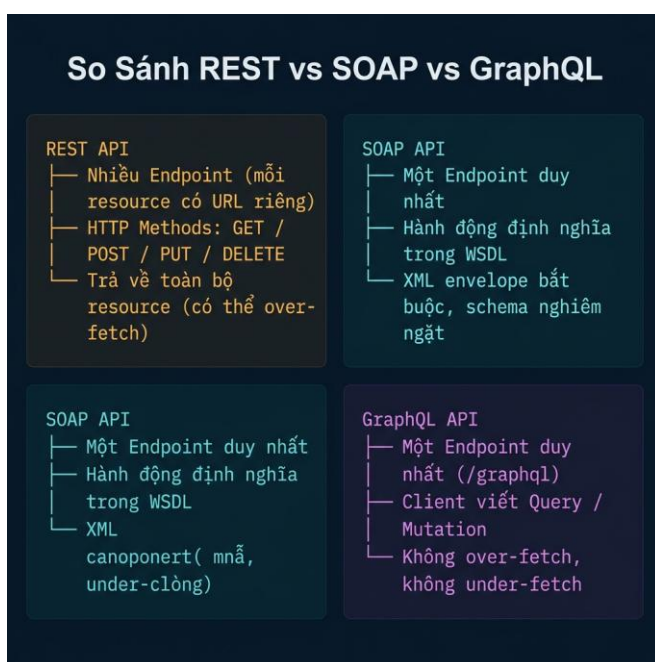
Lưu ý: YAML thường được dùng để *mô tả* API (OpenAPI/Swagger spec), **không** phải định dạng payload trong HTTP body của REST API thực tế.

3.2 Phân loại theo phạm vi truy cập

Loại	Mô tả
Public API	Mở cho mọi developer, thường có tài liệu công khai
Private API	Chỉ dùng nội bộ trong tổ chức, không expose ra ngoài

Loại	Mô tả
Partner API	Chia sẻ với đối tác kinh doanh được xác thực
Composite API	Kết hợp nhiều API, thực hiện nhiều bước trong một request

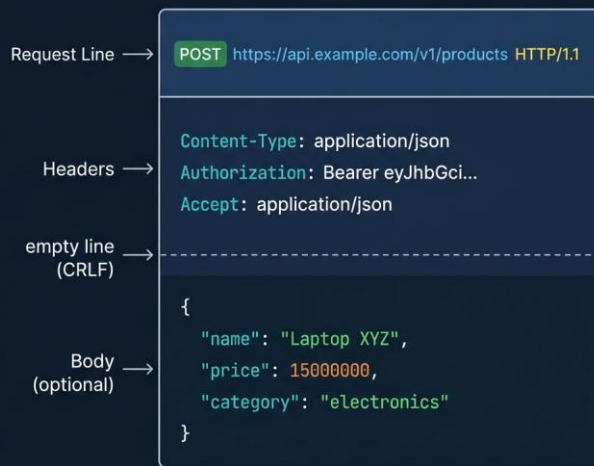
3.3 So sánh REST vs SOAP vs GraphQL



4. Cấu trúc HTTP Request

Một HTTP Request bao gồm các thành phần sau:

Cấu Trúc HTTP Request



Ví dụ thực tế:

```
POST https://api.example.com/v1/products HTTP/1.1
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Accept: application/json

{
  "name": "Laptop XYZ",
  "price": 15000000,
  "category": "electronics"
}
```

4.1 Các thành phần của Request

4.1.1 HTTP Method (Verb)

Xác định **hành động** muốn thực hiện lên resource (xem mục 6).

4.1.2 URL / Endpoint

Cấu Trúc URL / Endpoint

`https://api.example.com/v1/u1/users/42/status=pending&page=1`

`https://api.example.com/v1/users/42/orders?status=pending&page=1`

Base URL

Phần URL gốc chung

Path (Resource)

Đường dẫn tài nguyên

Query Parameters

Lọc và phân trang

- **Path Parameters:** Nhúng vào URL path — `/users/{id}` → `/users/42`
- **Query Parameters:** Thêm vào sau `?` — `?status=pending&page=1`
- **Fragment:** Phần sau `#`, không gửi đến server

4.1.3 Request Headers

Header	Ý nghĩa	Ví dụ
<code>Content-Type</code>	Định dạng của request body	<code>application/json</code>
<code>Accept</code>	Định dạng mong muốn nhận về	<code>application/json</code>
<code>Authorization</code>	Thông tin xác thực	<code>Bearer <token></code> / <code>Basic <b64></code>
<code>X-API-Key</code>	API Key xác thực	<code>X-API-Key: abc123xyz</code>

Header	Ý nghĩa	Ví dụ
User-Agent	Thông tin về client	PostmanRuntime/7.32.0
Accept-Language	Ngôn ngữ ưa thích	vi-VN, en-US
Cache-Control	Chính sách cache	no-cache
Cookie	Dữ liệu cookie gửi kèm	session_id=abc123

4.1.4 Request Body

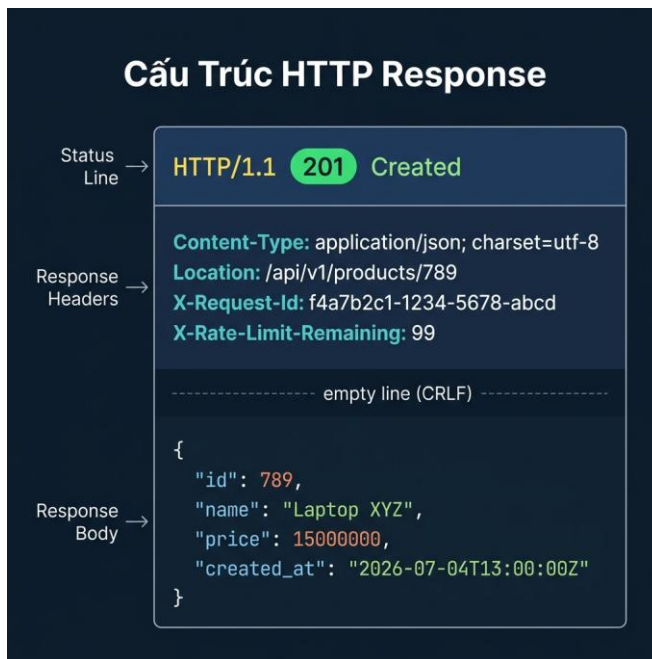
Dữ liệu gửi lên server, thường đi kèm với POST, PUT, PATCH:

```
{
  "username": "john_doe",
  "email": "john@example.com",
  "password": "Secure@123",
  "role": "admin"
}
```

Các định dạng phổ biến: **JSON**, **XML**, **form-data** (multipart), **x-www-form-urlencoded**

5. Cấu trúc HTTP Response

Một HTTP Response bao gồm:



Ví dụ thực tế:

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: https://api.example.com/v1/products/789
X-Request-Id: f4a7b2c1-1234-5678-abcd-ef0987654321

{
  "id": 789,
  "name": "Laptop XYZ",
  "price": 15000000,
  "category": "electronics",
  "created_at": "2026-07-04T13:00:00Z"
}
```

5.1 Các thành phần của Response

5.1.1 Status Code

Ba chữ số cho biết kết quả của request (xem chi tiết mục 7).

5.1.2 Response Headers

Header	Ý nghĩa	Ví dụ
Content-Type	Định dạng của response body	application/json; charset=utf-8
Content-Length	Kích thước body (bytes)	248
Location	URL của resource vừa được tạo (sau 201)	/api/users/123
Cache-Control	Chính sách cache phía client	max-age=3600, public
WWW-Authenticate	Thông báo yêu cầu xác thực (kèm 401)	Bearer realm="api"
X-Request-Id	ID định danh request (debug)	f4a7b2c1-...
X-Rate-Limit-*	Thông tin giới hạn tốc độ	X-Rate-Limit-Remaining: 99

5.1.3 Response Body

Dữ liệu trả về, thường là **JSON**:

```
{
  "status": "success",
  "data": {
    "user": {
      "id": 42,
      "name": "Nguyen Van A",
      "email": "a@example.com"
    }
  },
  "meta": {
    "timestamp": "2026-07-04T06:00:00Z",

```

```
    "version": "1.0"  
  }  
}
```

6. Các loại HTTP Methods

6.1 Bảng tổng hợp HTTP Methods

Method	Hành động	Idempotent	Safe	Có Body?	Mã thành công thường gặp
GET	Lấy dữ liệu resource	Có	Có	Không	200 OK
POST	Tạo mới resource	Không	Không	Có	201 Created
PUT	Cập nhật toàn bộ resource	Có	Không	Có	200 OK
PATCH	Cập nhật một phần resource	Phụ thuộc	Không	Có	200 OK
DELETE	Xóa resource	Có	Không	Tùy chọn	204 No Content
HEAD	Lấy headers (không có body)	Có	Có	Không	200 OK

Method	Hành động	Idempotent	Safe	Có Body?	Mã thành công thường gặp
OPTIONS	Liệt kê methods được hỗ trợ (CORS)	Có	Có	Không	200 OK / 204

Idempotent: Gọi nhiều lần có cùng kết quả như gọi một lần

Safe: Không thay đổi trạng thái server

PATCH & Idempotency: Theo RFC 5789, PATCH không được đảm bảo là idempotent theo thiết kế giao thức. Tuy nhiên, PATCH *có thể* là idempotent tùy vào implementation: nếu body chứa giá trị tuyệt đối (`{"email": "new@test.com"}`) → idempotent; nếu body chứa hành động tương đối (`{"increment_views": 1}`) → không idempotent.

6.2 Ví dụ sử dụng từng method

```
# GET – Lấy danh sách users
GET /api/v1/users?page=1&limit=10

# POST – Tạo user mới
POST /api/v1/users
Body: { "name": "Nguyen Van A", "email": "a@test.com" }

# PUT – Cập nhật toàn bộ thông tin user ID=42
PUT /api/v1/users/42
Body: { "name": "Nguyen Van B", "email": "b@test.com", "role": "admin" }

# PATCH – Chỉ cập nhật email
PATCH /api/v1/users/42
Body: { "email": "new@test.com" }

# DELETE – Xóa user ID=42
DELETE /api/v1/users/42
```

6.3 Mapping với CRUD

CRUD Operation	HTTP Method	SQL Equivalent
Create	POST	INSERT
Read	GET	SELECT
Update	PUT / PATCH	UPDATE
Delete	DELETE	DELETE

7. HTTP Status Codes

7.1 Phân nhóm tổng quan

Phân Nhóm HTTP Status Codes		
GRAY	1xx	Informational #2D3748 Request đã được nhận, đang tiếp tục xử lý 100 Continue 101 Switching Protocols
GREEN	2xx	Success ✓ #1B4332 Request thành công 200 OK 201 Created 204 No Content
YELLOW	3xx	Redirection #4A3300 Cần thêm hành động từ client 301 Moved 302 Found 304 Not Modified
ORANGE	4xx	Client Error ✗ #3D1A00 Lỗi từ phía client 400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found
RED	5xx	Server Error ✗ #3D0000 Lỗi từ phía server 500 Internal Server Error 503 Service Unavailable

7.2 Bảng chi tiết Status Codes

2xx — Thành công

Code	Tên	Ý nghĩa	Khi nào dùng
200	OK	Request thành công, có body trả về	GET, PUT, PATCH thành công
201	Created	Resource mới được tạo thành công	POST thành công
202	Accepted	Đã nhận request, đang xử lý async	Background job, queue
204	No Content	Thành công nhưng không có body	DELETE thành công
206	Partial Content	Trả về một phần dữ liệu	Download lớn, pagination

4xx — Lỗi Client

Code	Tên	Ý nghĩa	Khi nào gặp
400	Bad Request	Request sai cú pháp hoặc thiếu dữ liệu bắt buộc	Gửi sai format JSON, thiếu field

Code	Tên	Ý nghĩa	Khi nào gặp
401	Unauthorized	Chưa xác thực hoặc token không hợp lệ	Không gửi token, token hết hạn
403	Forbidden	Đã xác thực nhưng không có quyền	User không có quyền xóa bài người khác
404	Not Found	Resource không tồn tại	GET /users/9999 (không tồn tại)
405	Method Not Allowed	HTTP method không được phép	POST vào endpoint chỉ hỗ trợ GET
409	Conflict	Conflict với trạng thái hiện tại	Tạo user với email đã tồn tại
422	Unprocessable Entity	Cú pháp đúng nhưng dữ liệu không hợp lệ	Validation fails (email sai định dạng)
429	Too Many Requests	Vượt quá giới hạn rate limiting	Gọi API quá nhiều lần trong 1 phút

5xx — Lỗi Server

Code	Tên	Ý nghĩa	Khi nào gặp
500	Internal Server Error	Lỗi không xác định phía server	Bug trong code server
501	Not Implemented	Server chưa hỗ trợ chức năng này	Method chưa implement
502	Bad Gateway	Server nhận phản hồi không hợp lệ từ upstream	Proxy / Load balancer lỗi
503	Service Unavailable	Server quá tải hoặc đang bảo trì	Server down, maintenance
504	Gateway Timeout	Upstream server không phản hồi kịp thời	Database query timeout

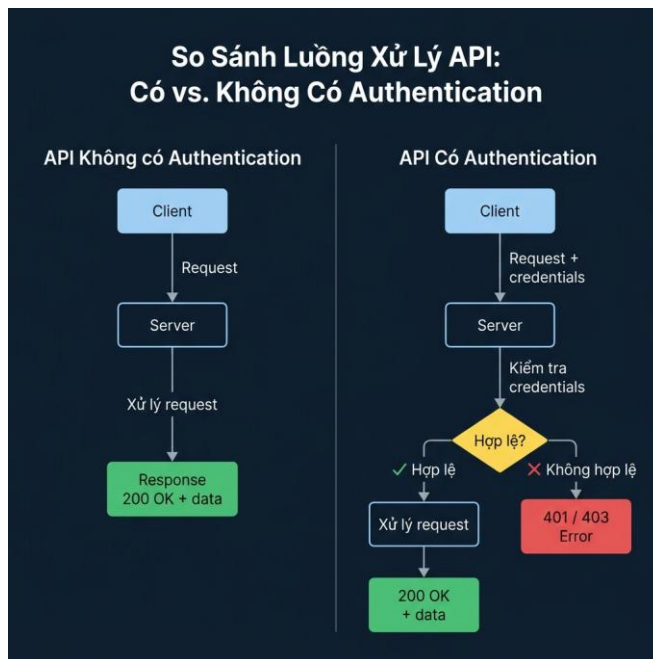
8. Phân biệt API có Authentication và không có Authentication

8.1 Tổng quan so sánh

Tiêu chí	API Không có Authentication	API Có Authentication
Định nghĩa	Endpoint công khai, không cần xác thực	Yêu cầu thông tin xác thực hợp lệ

Tiêu chí	API Không có Authentication	API Có Authentication
Ai có thể truy cập?	Bất kỳ ai biết URL	Chỉ những người có credentials hợp lệ
Thông tin gửi kèm	Không cần thêm gì	Token / Key / Username+Password
Rủi ro bảo mật	Cao — dễ bị abuse, scrape, DoS	Thấp hơn — có thể kiểm soát truy cập
Phù hợp cho	Public data, tài liệu công khai	Dữ liệu cá nhân, chức năng quan trọng
Ví dụ thực tế	Public weather API, Wikipedia API	Google Drive API, GitHub API
HTTP Status khi sai	N/A	401 Unauthorized hoặc 403 Forbidden

8.2 Sơ đồ luồng xử lý



8.3 Kiểm thử API Không có Authentication

Mục tiêu kiểm thử:

- Xác nhận endpoint trả về dữ liệu đúng mà **không cần gửi bất kỳ header xác thực nào**
- Kiểm tra rate limiting (nếu có)
- Kiểm tra hành vi khi gửi dữ liệu không hợp lệ

Test cases cần có:

TC ID	Mô tả	Input	Expected Output
TC-PUB-01	Gọi API không có header xác thực	<code>GET /api/public/products</code>	<code>200 OK</code> + data

TC ID	Mô tả	Input	Expected Output
TC-PUB-02	Gọi với tham số hợp lệ	GET /api/public/products?category=tech	200 OK + filtered data
TC-PUB-03	Gọi với tham số không hợp lệ	GET /api/public/products?page=-1	400 Bad Request
TC-PUB-04	Gọi với method không được hỗ trợ	DELETE /api/public/products	405 Method Not Allowed

8.4 Kiểm thử API Có Authentication

Test cases cần có:

TC ID	Mô tả	Input	Expected Output
TC-AUTH-01	Gọi API không có token	GET /api/orders (no headers)	401 Unauthorized
TC-AUTH-02	Gọi API với token hết hạn	Authorization: Bearer <expired_token>	401 Unauthorized

TC ID	Mô tả	Input	Expected Output
TC-AUTH-03	Gọi API với token không hợp lệ	Authorization: Bearer invalid123	401 Unauthorized
TC-AUTH-04	Gọi API với token hợp lệ	Authorization: Bearer <valid_token>	200 OK + data
TC-AUTH-05	User thường gọi endpoint admin	Bearer <user_token> + GET /admin/users	403 Forbidden
TC-AUTH-06	Admin gọi endpoint admin	Bearer <admin_token> + GET /admin/users	200 OK + data

8.5 Authentication vs Authorization — Sự khác biệt

Authentication vs Authorization — Sự Khác Biệt

Authentication — Xác thực



Bạn là ai?

- ☐ Credentials có đúng không?
- ☐ Token có hợp lệ không?
- ☐ Username/Password match?
- ☐ Credentials có runnec?
- ☐ Passworformation scan?
- ☐ Berneat di cinre role?

Lỗi khi thất bại

401 Unauthorized

Authorization — Phân quyền



Bạn được làm gì?

- ☐ Role/Permission của user
- ☐ Scope của token
- ☐ Access level check
- ☐ Permission của user
- ☐ Access level check
- ☐ Shield rsk scope check

Lỗi khi thất bại

403 Forbidden

9. Các loại Authentication trong API

9.1 Bảng so sánh tổng quát

Phương thức	Cơ chế hoạt động	Độ bảo mật	Độ phức tạp	Phù hợp cho
No Auth	Không cần xác thực	Rất thấp	Rất thấp	Public data, Open APIs
Basic Auth	Username:Password (Base64)	Thấp	Thấp	Legacy, internal tools
API Key	Static unique string	Thấp	Thấp	Public APIs, project-level
Bearer Token/JWT	Signed token	Cao	Trung	Modern web/mobile apps
OAuth 2.0	Delegated access protocol	Rất cao	Cao	Third-party integrations
mTLS	Certificate-based	Rất cao	Rất cao	Enterprise, microservices

9.2 No Authentication

- Mô tả:** Endpoint hoàn toàn công khai, không cần bất kỳ credential nào

- **Ví dụ:** GET `https://api.weather.gov/points/38.8894,-77.0352`
- **Rủi ro:** Dễ bị abuse, scraping, DoS

```
GET /api/public/weather?city=HoChiMinh HTTP/1.1
Host: api.example.com
```

9.3 Basic Authentication

- **Cơ chế:** Mã hóa `username:password` bằng Base64 và gửi trong header
- **Bắt buộc:** Phải dùng HTTPS (vì Base64 không phải mã hóa thực sự)
- **Header:** `Authorization: Basic dXN1cm5hbWU6cGFzc3dvcmQ=`

```
GET /api/protected/data HTTP/1.1
Host: api.example.com
Authorization: Basic dXN1cm5hbWU6cGFzc3dvcmQ=
```

`dXN1cm5hbWU6cGFzc3dvcmQ=` = `Base64("username:password")`

Điểm yếu: Nếu dùng HTTP (không có S), credential bị lộ hoàn toàn!

9.4 API Key

- **Cơ chế:** Server tạo một chuỗi unique, client gửi trong header hoặc query param
- **Đặc điểm:** Long-lived (tồn tại lâu dài), thường dùng để định danh **ứng dụng**

```
# Cách 1: Trong Header (khuyến nghị)
GET /api/data HTTP/1.1
X-API-Key: sk-1234567890abcdef
```

```
# Cách 2: Trong Query Parameter (kém an toàn hơn vì lộ trong URL log)
GET /api/data?api_key=sk-1234567890abcdef HTTP/1.1
```

Nhược điểm: Nếu key bị lộ, cần revoke và tạo lại thủ công.

9.5 Bearer Token (JWT)

- **Cơ chế:** Sau khi đăng nhập thành công, server cấp token. Client gửi token trong mỗi request.
- **JWT (JSON Web Token)** gồm 3 phần: `Header.Payload.Signature`

Cấu Trúc JWT (JSON Web Token)

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.

HEADER

{ "alg": "HS256", "typ": "JWT" }
Thuật toán ký + loại token

eyJzdWIiOiI0MiIsIm5hbWUiOiJBbWJ9.

PAYLOAD

{ "sub": "42", "name": "An", "iat": 1600000000 }
Dữ liệu người dùng (claims)

SfIKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV

SIGNATURE

HMACSHA256(base64(header) + '.' + base64(payload), secret)
Chữ ký xác minh tính toàn vẹn

⚠ Payload có thể decode được (Base64) — KHÔNG lưu thông tin nhạy cảm trong JWT!

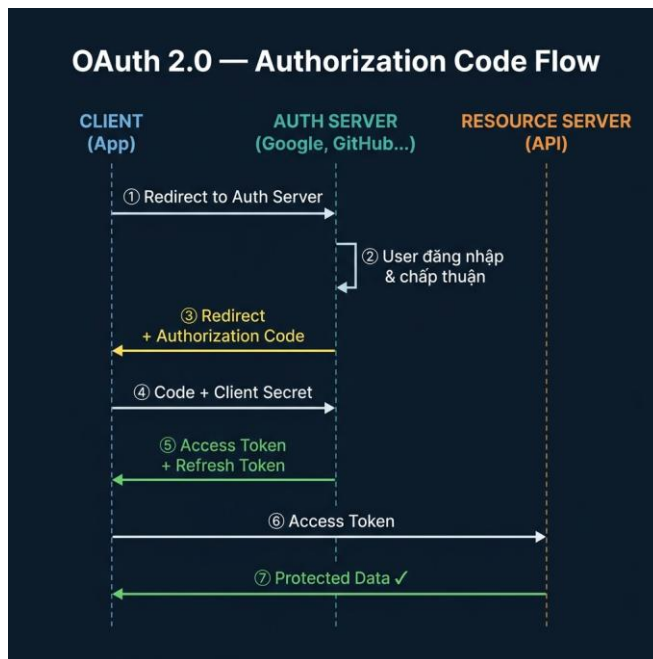
```
GET /api/v1/profile HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Ưu điểm: Stateless, có thể chứa thông tin user (claims), có expiration time (thường 15-60 phút).

9.6 OAuth 2.0

- **Cơ chế:** Giao thức ủy quyền cho phép ứng dụng bên thứ ba truy cập tài nguyên mà không cần mật khẩu người dùng
- **Ví dụ:** "Đăng nhập bằng Google" trên ứng dụng bên thứ ba

Luồng Authorization Code (phổ biến nhất):



Các Grant Types:

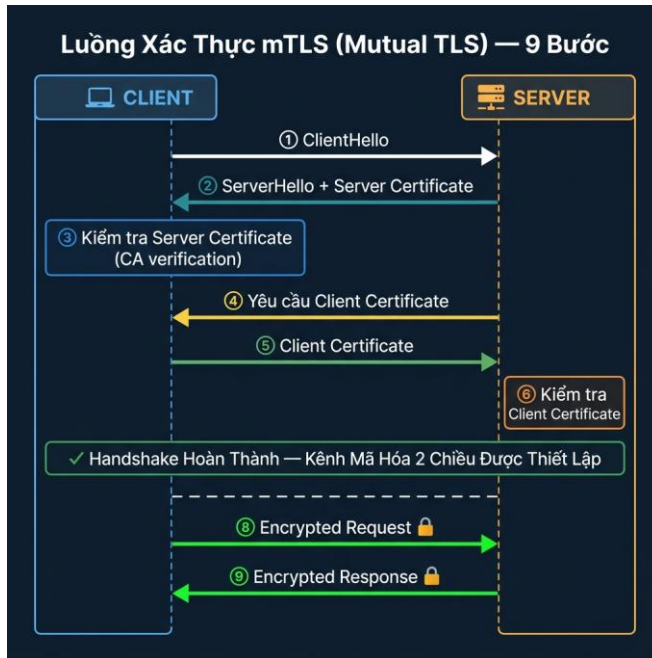
Grant Type	Dùng cho
Authorization Code	Web apps, Mobile apps (phổ biến nhất)
Client Credentials	Server-to-server (không có user)
Implicit	Legacy SPAs (deprecated)
Resource Owner Password	Legacy, không khuyến khích

9.7 mTLS (Mutual TLS)

- **Cơ chế:** Cả client **và** server đều phải cung cấp certificate để xác thực lẫn nhau — khác với TLS thông thường chỉ server xác thực với client

- **Phù hợp cho:** Hệ thống enterprise, microservices nội bộ, machine-to-machine (M2M) communication
- **Độ bảo mật:** Cao nhất — không thể giả mạo nếu không có certificate hợp lệ

Luồng xác thực mTLS:



Ví dụ cấu hình (curl):

```
curl --cert client.crt \
  --key client.key \
  --cacert ca.crt \
  https://internal-api.example.com/data
```

So sánh TLS vs mTLS:

Tiêu chí	TLS (1 chiều)	mTLS (2 chiều)
Server xác thực	Có	Có

Tiêu chí	TLS (1 chiều)	mTLS (2 chiều)
Client xác thực	Không	Có (bằng client certificate)
Phù hợp cho	Web/Mobile apps thông thường	Internal APIs, Microservices, IoT
Quản lý certificate	Chỉ server	Cả server và client
Overhead	Thấp	Cao hơn (quản lý cert phức tạp)

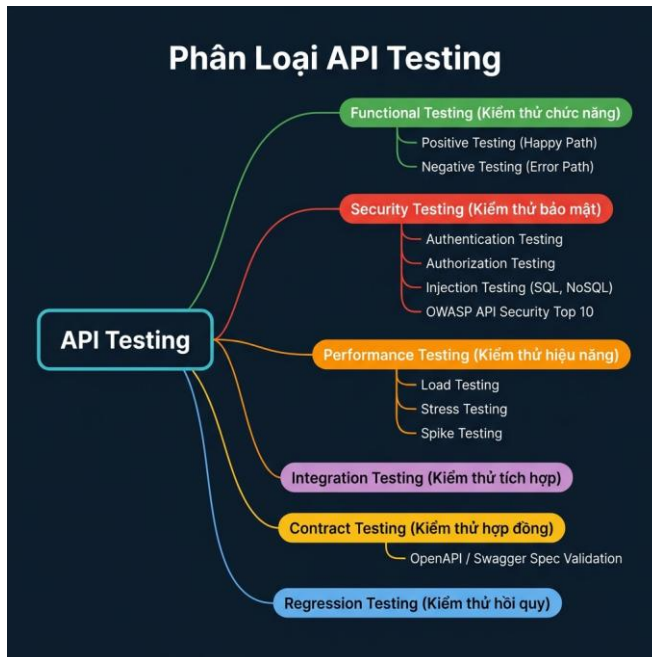
Lưu ý thực tế: mTLS phổ biến trong Kubernetes service mesh (Istio, Linkerd), API gateway nội bộ, và các hệ thống tài chính/ngân hàng yêu cầu bảo mật cao.

Nhược điểm:

- Phức tạp trong việc quản lý vòng đời certificate (rotation, revocation)
- Không phù hợp với API công khai vì đòi hỏi client phải có certificate
- Chi phí hạ tầng PKI (Public Key Infrastructure) cao

10. Phân loại API Testing

10.1 Theo mục đích kiểm thử



10.2 Functional Testing

Positive Testing (Happy Path):

- Gửi request với đầy đủ dữ liệu hợp lệ
- Xác nhận status code đúng (200, 201...)
- Xác nhận cấu trúc và giá trị response body
- Xác nhận headers phản hồi

Negative Testing (Error Path):

- Thiếu field bắt buộc → 400 Bad Request
- Sai kiểu dữ liệu (string thay vì integer) → 400/422
- Vượt quá giới hạn ký tự → 400/422
- Gửi null/empty → phải xử lý đúng
- Method không được phép → 405 Method Not Allowed

10.3 Security Testing (OWASP API Security Top 10 — 2023)

Nguồn: [OWASP API Security Top 10 \(2023\)](#)

#	Vulnerability	Mô tả	Test cần làm
1	Broken Object Level Authorization	Truy cập resource của user khác (IDOR)	Thay đổi ID trong URL
2	Broken Authentication	Token bypass, weak credentials	Test expired/invalid tokens
3	Broken Object Property Level Auth	Nhận field nhạy cảm không được phép	Kiểm tra response không lộ password
4	Unrestricted Resource Consumption	DoS qua request lớn	Fuzz testing với payload lớn
5	Broken Function Level Authorization	User thường gọi API admin	Test RBAC bypasses
6	Unrestricted Access to Sensitive Flows	Abuse business logic	Race conditions, mass enrollment
7	Server Side Request Forgery (SSRF)	API gọi đến URL do attacker kiểm soát	Gửi URL nội bộ trong body
8	Security Misconfiguration	Debug mode on, CORS wildcard	Kiểm tra headers, options
9	Improper Inventory Management	Shadow APIs, outdated endpoints	Kiểm tra tất cả endpoints

#	Vulnerability	Mô tả	Test cần làm
10	Unsafe Consumption of APIs	Tin tưởng mù quáng vào API bên thứ ba	Validate third-party responses

11. Checklist kiểm thử API

11.1 Kiểm thử cơ bản (mỗi endpoint)

- [] **Status Code** đúng với scenario (200, 201, 400, 401, 403, 404, 500...)
- [] **Response Body** đúng schema (đủ field, đúng kiểu dữ liệu)
- [] **Response Headers** có đầy đủ (Content-Type, security headers)
- [] **Thời gian phản hồi** trong ngưỡng chấp nhận được (< 2s cho API thông thường)
- [] **Dữ liệu nhất quán** với database sau request

11.2 Kiểm thử Authentication

- [] Không gửi token → 401
- [] Token sai → 401
- [] Token hết hạn → 401
- [] Token hợp lệ → 200 (và dữ liệu đúng)
- [] Token của user A dùng xem tài nguyên của user B → 403 hoặc 404

11.3 Kiểm thử Validation

- [] Thiếu field bắt buộc → 400/422
- [] Sai định dạng (email không có @) → 400/422
- [] Giá trị ngoài phạm vi (age = -1 hoặc 999) → 400/422
- [] Chuỗi quá dài → 400/422
- [] SQL Injection trong input → phải bị từ chối hoặc escape

- [] XSS trong input → phải bị sanitize

11.4 Kiểm thử Performance

- [] Response time < 500ms (95th percentile)
- [] API xử lý được số lượng concurrent users mong đợi
- [] Rate limiting hoạt động đúng (→ 429 khi vượt giới hạn)
- [] API không crash khi payload lớn

12. Công cụ kiểm thử API phổ biến

Công cụ	Loại	Ưu điểm	Dùng cho
Postman	GUI + Script	Dễ dùng, có Collection, Test script, Mock server	Manual + Automated testing
REST Assured	Java Library	Tích hợp với JUnit/TestNG, CI/CD friendly	Automated API testing (Java)
pytest + requests	Python	Linh hoạt, rich ecosystem	Automated API testing (Python)
curl	CLI	Lightweight, có sẵn trên Unix	Quick manual testing
k6	Performance	JS-based, load testing	Performance testing
JMeter	Performance	GUI + script, phổ biến	Load/Stress testing

Công cụ	Loại	Ưu điểm	Dùng cho
SoapUI	GUI	Chuyên SOAP, có assertions mạnh	SOAP API testing
Insomnia	GUI	Đơn giản hơn Postman, GraphQL support	Manual testing, GraphQL
Pact	Contract	Consumer-driven contract testing	Microservices contract testing